

Roteiro de Aula – Comparando Objetos

1. Ordenando objetos

Vamos criar a classe Pessoa, que representa uma pessoa com nome e dataNascimento. Ela implementa a interface Comparable<Pessoa>.

```
import java.time.LocalDate;

public class Pessoa implements Comparable<Pessoa> {
    private String nome;
    private LocalDate dataNascimento;

    public Pessoa(String nome, LocalDate dataNascimento) {
        this.nome = nome;
        this.dataNascimento = dataNascimento;
    }

    public String getNome() {
        return nome;
    }

    public LocalDate getDataNascimento() {
        return dataNascimento;
    }

    @Override
    public String toString() {
        return "Pessoa{" +
            "nome='" + nome + '\'' +
            ", dataNascimento=" + dataNascimento +
            '}';
    }

    //implementa comparable compareTo
    @Override
    public int compareTo(Pessoa outra) {
        return this.nome.compareTo(outra.nome);
    }

    @Override
    public int hashCode() {
        return nome.hashCode();
    }

    @Override
```

```
public boolean equals(Object obj) {  
    if (getClass() != obj.getClass() || this == obj)  
        return false;  
    Pessoa other = (Pessoa) obj;  
    if (nome == null) {  
        if (other.nome != null)  
            return false;  
    } else if (!nome.equals(other.nome))  
        return false;  
    return true;  
}  
  
}
```



1. **Para que serve a definição: implements Comparable<Pessoa> na declaração da classe Pessoa e o método compareTo()?**

Serve para comparar Pessoa com Pessoa de acordo com o atributo definido no método. Nesse caso do código acima está comparando nome.

Classe Main para Teste

Utilize a classe App no mesmo pacote da classe BubbleSort. O objetivo é testar ordenação crescente, decrescente, de objetos da classe pessoa.

```
public class App {  
    public static Pessoa[] vetorDePessoas() {  
        return new Pessoa[]{  
            new Pessoa("João", LocalDate.of(1990, 1, 1)),  
            new Pessoa("Ana", LocalDate.of(1992, 2, 2)),  
            new Pessoa("Maria", LocalDate.of(1995, 5, 5)),  
            new Pessoa("Pedro", LocalDate.of(1993, 4, 4)),  
            new Pessoa("Lucas", LocalDate.of(1991, 6, 6)),  
            new Pessoa("Carlos", LocalDate.of(1988, 3, 3))  
        };  
    }  
  
    public static void main(String[] args) {  
        BubbleSort<Pessoa> bubbleSortPessoas = new BubbleSort<>();  
        Pessoa[] pessoas = vetorDePessoas();  
    }  
}
```

```
        bubbleSortPessoas.sort(pessoas);  
        System.out.println(Arrays.toString(pessoas));  
    }  
}
```

Comparator<T> – ordem externa (fora da classe)

- É uma interface usada para criar diferentes critérios de comparação, fora da classe.
- Útil quando você não pode ou não quer alterar a classe original ou deseja várias formas de ordenar.

Vamos criar um ordenador externo, apenas por data, para isso crie a classe

```
public class ComparadorData implements Comparator<Pessoa> {  
    @Override  
    public int compare(Pessoa p1, Pessoa p2) {  
        return p1.getDataNascimento().compareTo(p2.getDataNascimento());  
    }  
}
```

2.  Crie a instancia da class ComparadorData no main e passe o mesmo como parâmetro para o método sort. Apresente o código abaixo.

```
ComparadorData comparadorData = new ComparadorData();  
  
bubbleSortPessoas.sort(pessoas, comparadorData);
```

Também é possível usar uma expressão lambda para criar o comparador.

A expressão lambda é uma forma mais curta de criar um objeto que implementa Comparator<Pessoa>, sem precisar declarar uma nova classe. Como Comparator possui o método compare, a lambda informa diretamente como duas pessoas devem ser comparadas.

Exemplo:

```
Comparator<Pessoa> comparadorNomeData = (p1, p2) -> {  
    int compNome = p1.getNome().compareTo(p2.getNome());  
    if (compNome != 0) {  
        return compNome;  
    }  
    return p1.getDataNascimento().compareTo(p2.getDataNascimento());  
};
```

Nesse caso, p1 e p2 são os dois objetos Pessoa que estão sendo comparados. Primeiro o código compara os nomes; se forem iguais, compara as datas de nascimento.



3. **Adicione a expressão lambda direto no main, depois passe o objeto para sort. Apresente o resultado após adicionar uma pessoa com nome repetido com data de nascimento diferente.**

Pessoas ordenadas:

Nome='Ana', DataNascimento=1985-02-02
Nome='Ana', DataNascimento=1992-02-02
Nome='Carlos', DataNascimento=1988-03-03
Nome='João', DataNascimento=1990-01-01
Nome='Lucas', DataNascimento=1991-06-06
Nome='Maria', DataNascimento=1995-05-05
Nome='Pedro', DataNascimento=1993-04-04



4. **Qual é a complexidade do algoritmo de ordenação Bubble Sort? O que isso quer dizer na prática?**

O (n²)
dado a entrada do n, quantas operações ele vai fazer.

Adicione o método de gerar dados aleatório para o vetor

```
private static Integer[] gerarVetorAleatorio(int n) {  
    Random random = new Random();
```

```
Integer [] vetor = new Integer [n];

for (int i = 0; i < n; i++) {
    vetor[i] = random.nextInt(n * 10);
}

return vetor;
}
```

Código para medir o tempo de execução

```
long inicio = System.currentTimeMillis();
// ordenar aqui
    long fim = System.currentTimeMillis();
    System.out.println("Tempo = " + (fim - inicio) + " ms");
```



5. **Leia o tamanho do vetor, gere e ordene com diferentes tamanhos. Meça o tempo de execução de cada um e apresente abaixo. O tempo cresce linearmente dado a entrada?**

50.000 - 16366 ms
100.000 - 56153 ms
200.000 - 228041 ms